

InterWasm

An Interpreter for a Subset of WebAssembly written in Java

WebAssembly



- WebAssembly (abbreviated Wasm) is a binary instruction format for a stack-based virtual machine.
- Wasm enables deployment of native applications (written in C, C++ or Rust) on web clients or servers
- Wasm aims execution of programs at near-native speed
- Wasm has a text format and a binary format

wat - WebAssembly Text Format

Example Wasm module with one function

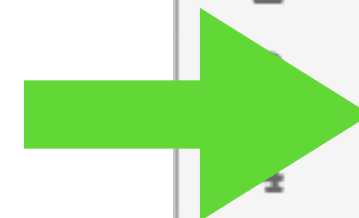
Function Type (Type Section)



WAT example: simple

```
1 (module
2   (func (export "addTwoAndConst") (param i32 i32) (result i32)
3     (local i32)
4     i32.const 5
5     local.set 2
6     local.get 0
7     local.get 1
8     i32.add
9     local.get 2
10    i32.add))
```

Function Body (Code Section)



wat - WebAssembly Text Format

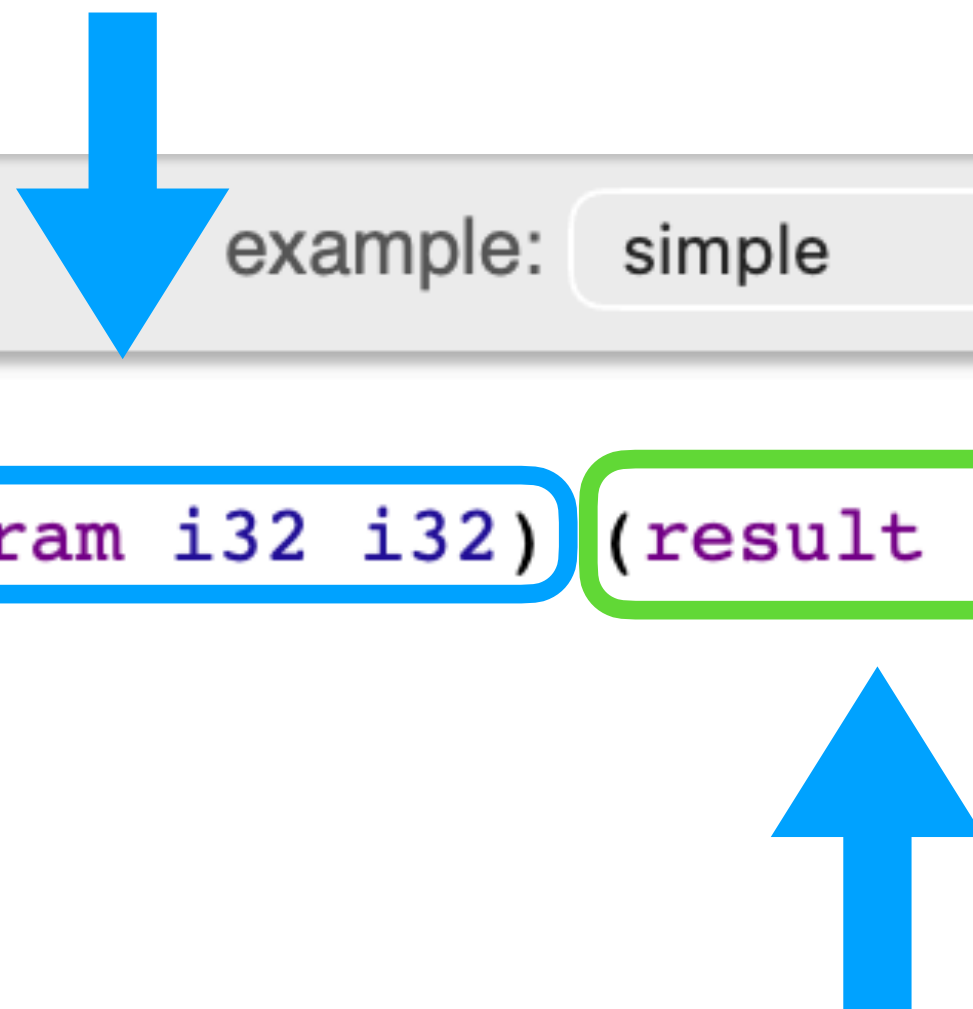
Function Type

Parameter definition

WAT

example: simple

```
1 (module
2   (func (export "addTwoAndConst") (param i32 i32) (result i32)
3
```



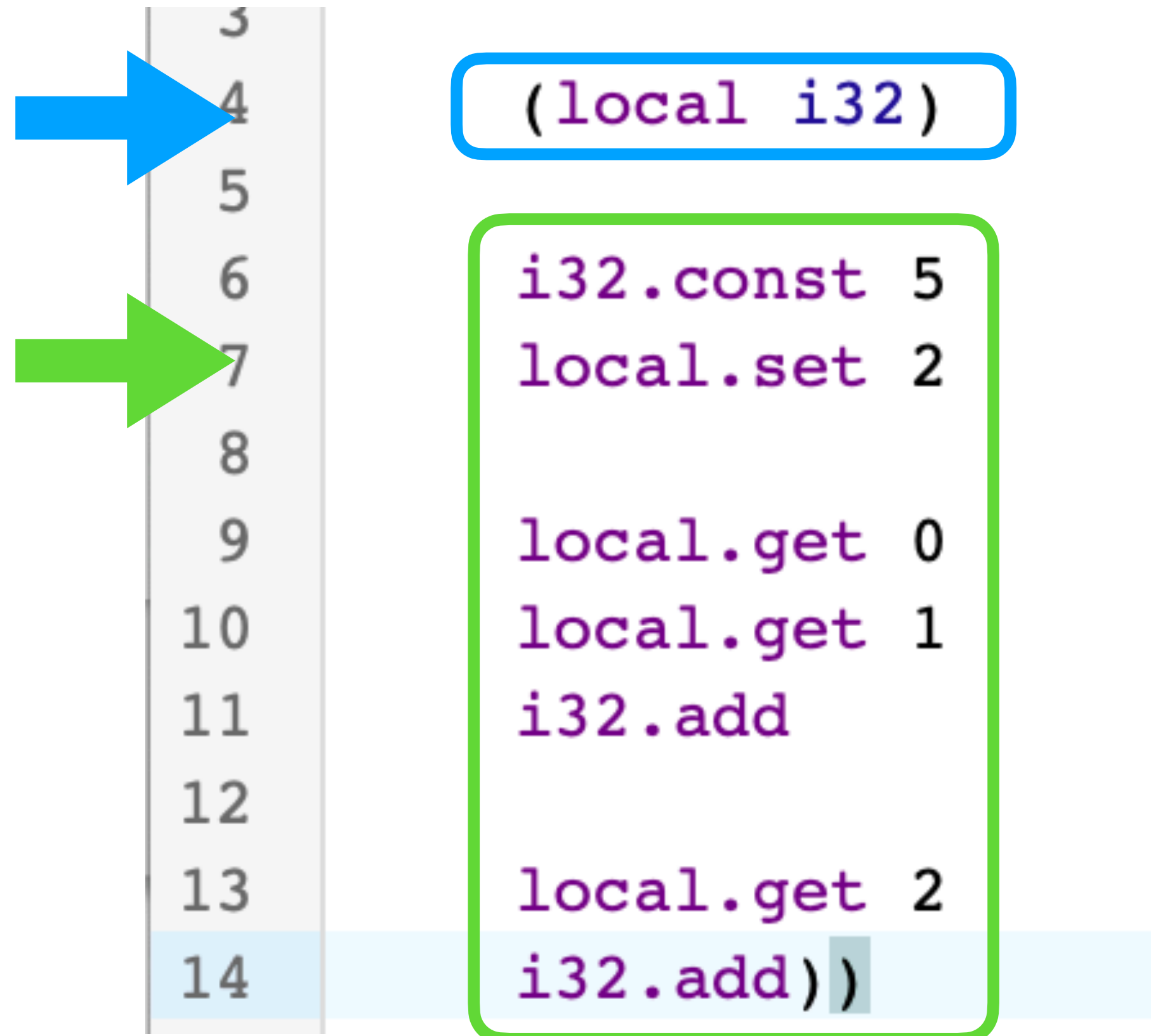
Definition of return values

wat - WebAssembly Text Format

Function body

Definition of Locals

Instructions



WebAssembly Basics

- Function arguments are stored as locals
- A function can have multiple return values
- Data Types
 - i32 - 32-bit integer
 - i64 - 64-bit integer
 - f32 - 32-bit float
 - f64 - 64-bit float
 - funcref - reference to other function
 - externref - reference to external value
- WebAssembly is based on multiple stack-like structures
 - Operand Stack: Typed stack for operations (where all data types can be stored e.x. i32, f64, ...)
 - Call Stack: Function calls create frames containing locals, parameters, return addresses, and so on.
 - Control Stack: Blocks, loops, and ifs have structured control frames, where input/output types are validated

WAT example: simple

```
1  (module
2    (func (export "addTwoAndConst") (param i32 i32) (result i32)
3      (local i32)
4
5      i32.const 5
6      local.set 2
7
8      local.get 0
9      local.get 1
10     i32.add
11
12     local.get 2
13     i32.add))
14
```

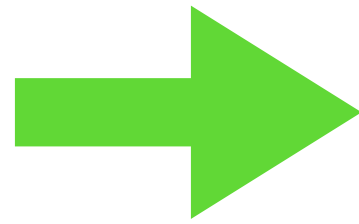
wasm - WebAssembly Bytecode

- The WebAssembly Bytecode
 - starts with the magic bytes 0061 736d 0100 0000 (.asm....)
 - is structured in up to 13 sections. Relevant for our interpreter:
 - Type Section (ID 1) - contains the function type for all functions
 - Code Section (ID 10) - contains the function body for all functions
- The ordering of the entries in the Type and Code section are used for mapping, e.x. function type 0 has function body 0

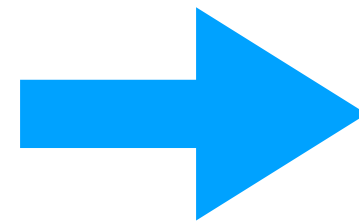
wasm- WebAssembly Bytecode

Bytecode of the shown example module

Magic Bytes



Type Section



...

```
00000000: 0061 736d                                ; WASM_BINARY_MAGIC
00000004: 0100 0000                                ; WASM_BINARY_VERSION

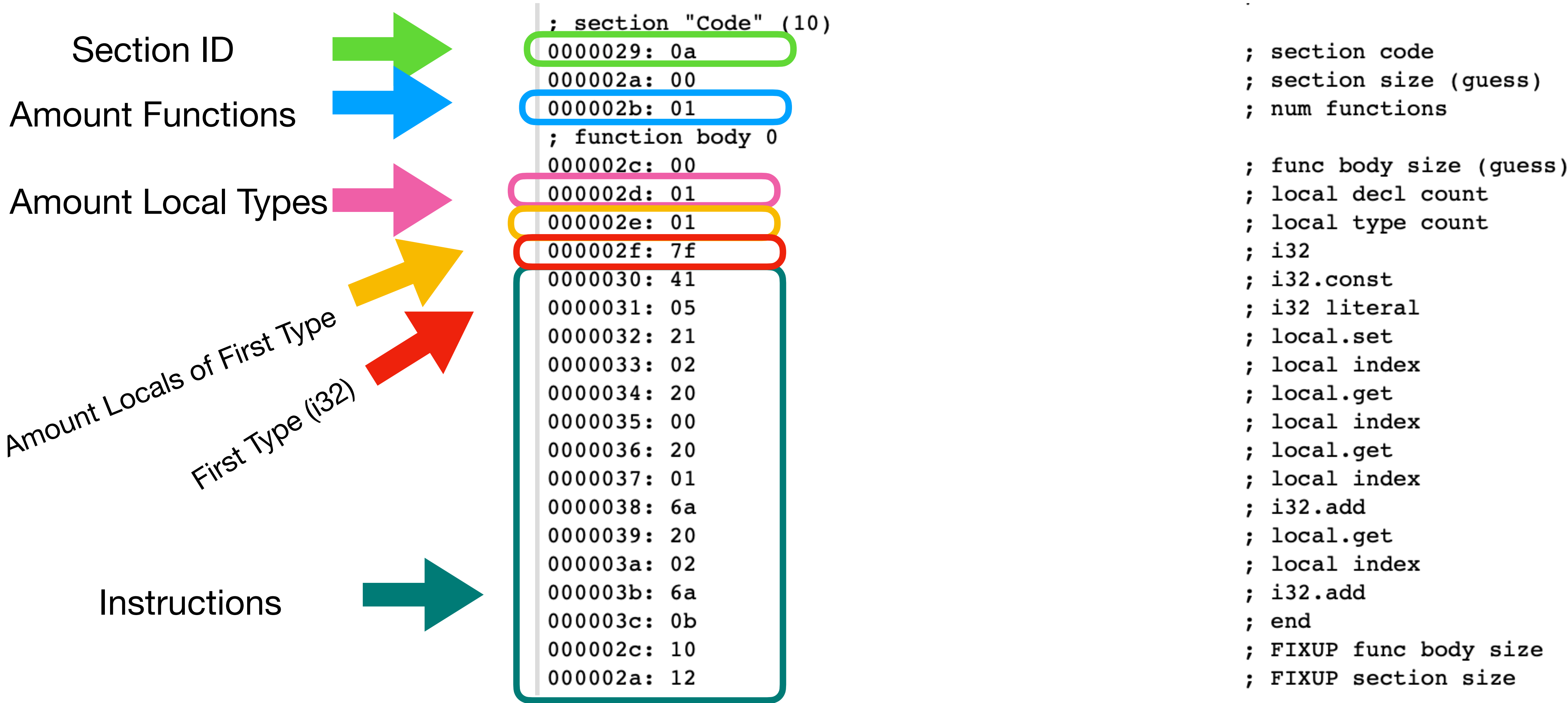
; section "Type" (1)
00000008: 01                                         ; section code
00000009: 00                                         ; section size (guess)
0000000a: 01                                         ; num types
; func type 0
0000000b: 60                                         ; func
0000000c: 02                                         ; num params
0000000d: 7f                                         ; i32
0000000e: 7f                                         ; i32
0000000f: 01                                         ; num results
00000010: 7f                                         ; i32
00000009: 07                                         ; FIXUP section size

; section "Function" (3)
00000011: 03                                         ; section code
00000012: 00                                         ; section size (guess)
00000013: 01                                         ; num functions
00000014: 00                                         ; function 0 signature index
00000012: 02                                         ; FIXUP section size

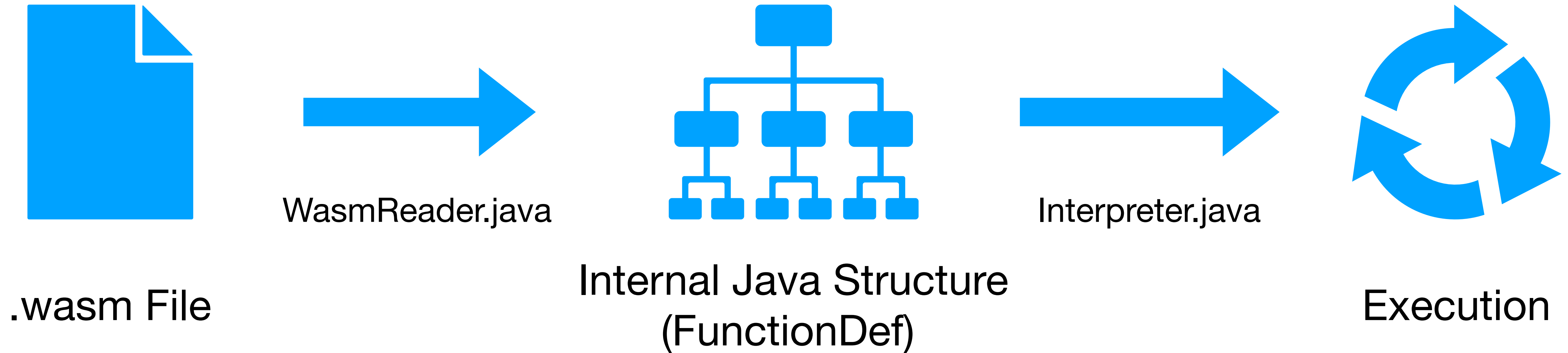
; section "Export" (7)
00000015: 07                                         ; section code
00000016: 00                                         ; section size (guess)
00000017: 01                                         ; num exports
```

wasm- WebAssembly Bytecode

Bytecode of the code section



InterWasm Pipeline



FunctionDef

```
// 2+3
FunctionDef fn = new FunctionDef(
    new FunctionType(
        List.of(), // no params
        ValueType.I32 // result is integer 32
    ),
    List.of(), // no locals
    // Instructions
    List.of(
        new ConstI32(2),
        new ConstI32(3),
        new I32Add()
    )
);
```

InterWasm - WebAssembly Subset

- InterWasm supports
 - .wasm files with up to one function
 - Locals
 - Passing function arguments
 - One return value
- InterWasm does not support
 - Host-Functions or imports (e.g. printf)
 - Memory (storage of raw bytes)
- InterWasm only supports a subset of the WebAssembly Instructions (next slide)

Supported WebAssembly Subset

- Data Types
 - i32
- Constant Operations
 - i32.const
- Arithmetic Operations
 - i32.add
 - i32.divs
 - i32.mul
 - i32.rems
 - i32.sub
- i32 Comparison Operations: eq, eqz, ges, gts, les, lts, ne
- Locals Operations
 - local.get
 - local.set
 - local.tee
- Control Flow Operations
 - block
 - loop
 - return
 - if, else end, brif, br

Benchmarking

- Counting primes algorithm
- Performed with inputs
 - 10 000
 - 100 000
 - 1 000 000

```
1  function _start(n):
2      // Count how many prime numbers are <= n
3
4      i = 2
5      count = 0
6
7      while i <= n:
8          is_prime = true
9          d = 2
10
11         while d * d <= i:
12             if i % d == 0:
13                 is_prime = false
14                 break
15
16             d = d + 1
17
18         if is_prime:
19             count = count + 1
20
21         i = i + 1
22
23     return count
```

Pseudocode

Performance Comparison

- TODO